

Phân Tích Thuật Toán - Dự Án Gomoku AI (Caro Trí Tuệ Nhân Tạo)

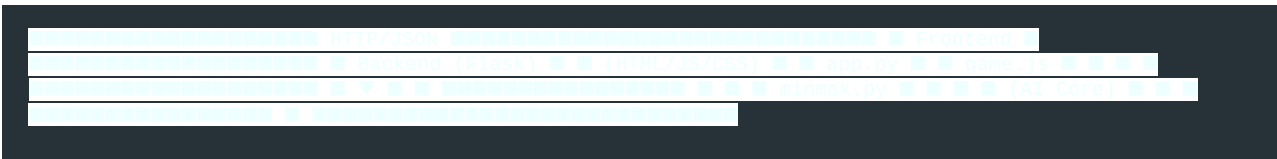
Mục Lục

- [Tổng Quan Dự Án](#)
- [Thống Kê & Giới Thích Về Các Thuật Toán](#)
- [Xếp Hạng Độ Khó \(Dễ → Khó\)](#)
- [Phân Chia Công Việc Cho 6 Người](#)
- [Góc Nhìn Giảng Viên – Các Điểm Cần Phân Tích Sâu & Câu Hỏi Thường Gặp](#)

I. Tổng Quan Dự Án

Gomoku AI là ứng dụng web chơi Caro (Gomoku) trên bàn cờ 15x15, tích hợp trí tuệ nhân tạo (AI) sử dụng thuật toán Minimax kết hợp Alpha-Beta Pruning và Iterative Deepening.

Kiến trúc tổng thể:



Danh sách các file mã nguồn chính:

File	Dòng code	Vai trò
minimax.py	380	Bộ não AI: chứa toàn bộ thuật toán cốt lõi
app.py	150	Backend server Flask, quản lý trạng thái game
game.js	280	Frontend logic, tương tác UI bàn cờ

II. Thống Kê & Giới Thích Về Các Thuật Toán

Thuật toán 1: Kiểm tra điều kiện thắng (Check Winner)

Thuộc tính	Chi tiết
File	minimax.py — dòng 14–34

Hàm	check_winner(board, player)
Độ khó	Trung bình

Mô tả: Quét toàn bộ bàn cờ 15x15 theo 4 hướng (ngang, dọc, chéo chính, chéo phụ). Tìm kiếm ô, kiểm tra liệu có đúng 5 quân liên tiếp cùng màu hay không.

Thuật toán cốt lõi: Brute-force scan với vòng lặp ba tầng: - Tầng 1: Duyệt tất cả 225 ô (15x15). - Tầng 2: Tìm kiếm ô, duyệt 4 hướng [(1,0), (0,1), (1,1), (1,-1)]. - Tầng 3: Kiểm tra 5 ô liên tiếp theo hướng đó.

Phức tạp: $O(N^2 \times 4 \times 5) = O(N^2)$ với $N = 15 \rightarrow$ tối đa 4.500 phép kiểm tra.

Thuật toán 2: Tìm mẫu chuỗi con (Pattern Counting)

Thuộc tính	Chi tiết
File	minmax.py — dòng 247–257
Hàm	<code>_count_pattern(s, pattern)</code>
Độ khó	Trung bình

Mô tả: Tìm số lần mẫu chuỗi con (pattern) xuất hiện trong chuỗi s bằng thuật toán tìm kiếm tuần tự (Sequential Search). Cho phép các lần xuất hiện **chồng lấn (overlapping)** vì con trỏ start chỉ tăng 1 sau mỗi lần tìm thấy.

Ví dụ: `_count_pattern("_XXX_XXX_", "_XXX_")` → trả về 2.

Vai trò: Là hàm nền tảng phục vụ cho hàm đánh giá thế cờ (`_evaluate_line`), tìm số lần xuất hiện các pattern 3 quân (3 quân, 4 quân, ...) trên mỗi đường thẳng.

Thuật toán 3: Trích xuất tất cả đường thẳng trên bàn cờ (Line Extraction)

Thuộc tính	Chi tiết
File	minmax.py — dòng 203–244
Hàm	<code>_extract_all_lines(board)</code>
Độ khó	Trung bình – Dễ

Mô tả: Trích xuất tất cả các "đường thẳng" (lines) trên bàn cờ theo 4 hướng: 1. **15 hàng ngang** (mỗi hàng 15 ô). 2. **15 cột dọc** (mỗi cột 15 ô). 3. **29 đường chéo chính** (top-left → bottom-right), chỉ giữ đường ≥ 5 ô. 4. **29 đường chéo phụ** (top-right → bottom-left), chỉ giữ đường ≥ 5 ô.

Tổng số đường: $15 + 15 + 21 + 21 =$ khoảng **72 đường thẳng**.

Vai trò: Cung cấp dữ liệu vào cho hàm `evaluate()`. Mỗi đường thẳng chỉ cần trích **đúng 1 lần** để tránh đếm trùng pattern (rất quan trọng cho tính chính xác của hàm đánh giá).

Thuật toán 4: Phát hiện chuỗi nguy hiểm (Danger Detection)

Thuộc tính	Chi tiết
File	minmax.py — dòng 39–61
Hàm	<code>is_dangerous(board, i, j)</code>
Độ khó	Trung bình – Dễ

Mô tả: Kiểm tra xem ô (i, j) (đã có quân) có nằm trong một chuỗi ≥ 2 quân liên tiếp cùng màu theo bất kỳ hướng nào hay không. Thuật toán tìm ô trung tâm ra 2 phía (tiền và lùi theo mỗi hướng, tối đa 3 bước mỗi phía).

Vai trò: Phức vụ cho hàm `get_possible_moves()`. Các ô lân cận khu vực "nguy hiểm" sẽ được ưu tiên xét trừ (must-block moves), giúp AI phản ứng nhanh với các đe dọa.

Thuật toán 5: Sinh các di chuyển viên (Move Generation)

Thuộc tính	Chi tiết
File	minmax.py — dòng 64–92
Hàm	<code>get_possible_moves(board)</code>
Độ khó	Trung bình

Mô tả: Thay vì xét tất cả 225 ô trống (rất lãng phí), thuật toán chỉ **sinh các di chuyển viên nằm trong bán kính 2 ô** xung quanh các quân của cả hai bên. Đây là kỹ thuật **Neighborhood-based Move Generation** phổ biến trong game AI.

Cấp độ ưu tiên: - **must_block**: Các ô nằm gần chuỗi nguy hiểm (≥ 2 quân liên tiếp) $\rightarrow$ ưu tiên cao. - **normal**: Các ô lân cận quân của bên mình $\rightarrow$ ưu tiên thấp hơn.

Kết quả: Danh sách di chuyển `combined = must_block + normal`, thường chỉ khoảng **20–60 di chuyển** thay vì 225.

Thuật toán 6: Đánh giá theo dòng (Line Evaluation / Heuristic Scoring)

Thuộc tính	Chi tiết
File	minmax.py — dòng 142–200
Hàm	<code>_evaluate_line(line)</code>
Đoạn mã	Dòng 150–170
Độ khó	Trung bình

Mô tả: Đây là **hàm heuristic cốt lõi** của AI. Mỗi dòng (line) trên bàn chơi được chuyển thành chuỗi ký tự (X = AI, O = Human, $_$ = trống), sau đó tìm kiếm xu hướng hình thành các **mẫu đe dọa (threat patterns)** và nhân với trọng số thích hợp.

Đánh pattern và trọng số:

Pattern	Ý nghĩa	Điểm AI (Tích cực)	Điểm Human (Phòng thủ)
---------	---------	--------------------	------------------------

XXXXX / 00000	5 liên tiếp (Thắng)	+10.000.000	-10.000.000
XXXX / _0000_	4 h 2 u	+1.000.000	-2.000.000
XXXX_ / 0000_	4 b 1 u	+80.000	-1.500.000
XX_XX / 00_00	4 gãy (broken 4)	+80.000	-1.500.000
XXX / _000_	3 h 2 u	+30.000	-60.000
_XX_X_ / _00_0_	3 gãy (broken 3)	+25.000	-55.000
XX / _00_	2 h	+500	-400

Chiến lược scoring bí mật: - Phòng thủ > Tấn công clip 4: `_HU_HALF_4 = 1.500.000 > _AI_OPEN_4 = 1.000.000`.
Mình này bu c AI chọn e d 4 quân c a i th tr c khi t t o chu i 4 h. - Tấn công > Phòng thủ clip 2-3 th p:
AI ch i ng t o th khi không b e d a tr c ti p.

Thuật toán 7: Hàm đánh giá toàn cục (Global Board Evaluation)

Thuộc tính	Chi tiết
File	minmax.py — dòng 260-292
Hàm	<code>evaluate(board)</code>
Độ khó	Trung bình

Mô tả: Đánh giá tổng thể trạng thái toàn bàn c, bao gồm 3 thành phần:

Pattern scoring: Gọi `_evaluate_line()` trên tất c ~72 hàng th ng (qua `_extract_all_lines()`).

Fork detection (Phát hiện th g ng kìm): Tìm s ng e d a clip cao (4 quân ho c 3 h). Nếu m t bên có ≥ 2 ng e d a ng th i $\rightarrow$ c ng/tr bonus l n (`100.000 + count \times 30.000`). Đây là k thuật nh n d i n **Double Threat / Fork** — th c g n nh không th ch n.

Center bonus (Th ng v trí trung tâm): Quân n m g n tâm bàn c (ô 7,7) c ng thêm i m theo công th c `max(0, 14 - manhattan_distance) \times 5`. Đây là heuristic ph b i n trong c Caro: quân trung tâm có nhi u h ng m r ng h n quân góc/biên.

Thuật toán 8: S p x p n c i (Move Ordering)

Thuộc tính	Chi tiết
File	minmax.py — dòng 96-139
Hàm	<code>score_move_quick(board, move, player)</code>
Độ khó	Khó

Mô t: Kỹ thuật **Move Ordering** là yếu tố quy định hiệu quả của Alpha-Beta Pruning. Trừ khi quy sâu, thuật toán đánh giá nhanh (lightweight evaluation) đang nên là phương pháp **Delta Evaluation (ánh giá chênh lệch cục bộ)**:

- Tính điểm **trước khi quân** (score_before) bằng cách quét 4 hàng ngang qua ô trống (mỗi hàng lấy 11 ô: $k = -5$ đến $k = +5$).
- Quân gì lép** vào ô trống.
- Tính điểm **sau khi quân** (score_after) bằng cách quét lại 4 hàng ngang.
- Tính $diff = score_after - score_before$.
- Hoàn tác** quân (quân là EMPTY).

Ý nghĩa diff: - $diff > 0$ (lớn): Nên chơi tiếp cho AI (tạo pattern tấn công hoặc phá pattern của đối thủ). - $diff < 0$ (nhỏ): Nên chơi tiếp cho Human.

Sắp xếp trong minimax: - Với AI (maximize): `moves.sort(reverse=True)` — nên có diff lớn nhất lên đầu. - Với Human (minimize): `moves.sort(reverse=False)` — nên có diff nhỏ nhất lên đầu.

Thuật toán 9: Minimax với Alpha-Beta Pruning

Thuộc tính	Chi tiết
File	minimax.py — dòng 300–361
Hàm	<code>minimax(board, depth, alpha, beta, maximizing, start_time, time_limit)</code>
Độ khó	★★★★★ Rất Khó (Thuật toán trung tâm)

Mô t: Đây là thuật toán **chống lại nhau** của toàn bộ đối phương. Minimax là thuật toán tìm kiếm **đối kháng** (Adversarial Search) trong lý thuyết trò chơi, nên có thể coi là kỹ thuật **Alpha-Beta Pruning**.

Nguyên lý Minimax: - Xây dựng **cây trò chơi (Game Tree)** với 2 loại nút xen kẽ: - **Maximizing node (AI):** Chọn nước đi có giá trị **cao nhất**. - **Minimizing node (Human):** Chọn nước đi có giá trị **thấp nhất**. - Từ lá cây (depth = 0 hoặc thắng/thua), trả về giá trị `evaluate(board)`.

Alpha-Beta Pruning: - α : Giá trị **thấp nhất** mà Maximizer (AI) đã tìm được. - β : Giá trị **cao nhất** mà Minimizer (Human) đã tìm được. - **Cắt tỉa:** Khi $\beta \leq \alpha$, không cần xét các nhánh còn lại vì chúng **không thể ảnh hưởng** đến quyết định cuối cùng. Trong trường hợp lý tưởng (move ordering hoàn hảo), Alpha-Beta giảm số nút xét từ $O(b^d)$ xuống $O(b^{d/2})$, tức giảm đôi độ sâu tìm kiếm.

Các kỹ thuật tối ưu bổ sung trong cài đặt: 1. **Depth-adjusted win score:** `WIN_SCORE + depth * 1000` — ưu tiên thắng nhanh hơn (thắng ở depth sâu hơn có giá trị cao hơn vì depth lớn = gần game hơn). 2. **Move pruning:** Cắt gọt chỉ giữ **20 nước đi tốt nhất** (`moves = moves[:20]`) sau khi sắp xếp, giảm branching factor đáng kể. 3. **Timeout check:** Kiểm tra thời gian tại mỗi node, ném `TimeoutException` để thoát sớm.

Thuật toán 10: Iterative Deepening (Tìm kiếm sâu dần)

Thuộc tính	Chi tiết
File	minimax.py — dòng 364–380

Hàm	<code>iterative_deepening(board, max_depth, time_limit)</code>
Độ khó	★★★★ Khó

Mô tả: Thay vì gọi Minimax trực tiếp ở độ sâu tối đa (có thể mất rất lâu hoặc không kịp gọi hàm thời gian), Iterative Deepening chạy Minimax lặp lại với độ sâu tăng dần: 1, 2, 3, ..., `max_depth`.

Cách hoạt động:

```
depth = 1 -> Minimax(depth=1) -> Lưu best move
depth = 2 -> Minimax(depth=2) -> Lưu best move
...
Tối ưu nhất hoàn chỉnh
```

Ưu điểm: 1. **Anytime Algorithm:** Luôn có kết quả hợp lệ bất kỳ lúc nào dừng. 2. **Thời gian tối ưu:** AI suy nghĩ sâu nhất có thể trong giới hạn thời gian (Easy: 0.5s, Medium: 1s, Hard: 3s). 3. **Không lãng phí:** Dù chạy nhiều lần, tổng số node xét thực chỉ chạy 1 lần ở độ sâu cuối cùng (do tính chất hàm đệ quy).

III. Xếp Hàng Độ Khó (Dễ → Khó)

Hạng	Thuật toán	Độ khó	Lý do
1	Check Winner (Kiểm tra thắng)	★	Vòng lặp đơn giản, dễ hiểu
2	Pattern Counting (Đếm mẫu)	★	Tìm kiếm chuỗi con cơ bản
3	Line Extraction (Trích xuất đường)	★★	Xử lý toạ độ đường chéo hơi phức tạp
4	Danger Detection (Phát hiện nguy hiểm)	★★	Quét 2 chiều theo 4 hướng
5	Move Generation (Sinh nước đi)	★★	Bán kính lân cận + phân loại ưu tiên
6	Line Evaluation (Đánh giá theo dòng)	★★★	Hệ thống pattern + trọng số phức tạp
7	Global Evaluation (Đánh giá toàn cục)	★★★	Tích hợp 3 thành phần: pattern + fork + center
8	Move Ordering (Sắp xếp nước đi)	★★★★	Delta evaluation + hiểu rõ Minimax
9	Iterative Deepening (Sâu dần)	★★★★	Quản lý thời gian + exception flow
10	Minimax + Alpha-Beta Pruning	★★★★★	Quy tắc kháng + cắt tỉa + tối ưu nhiều tầng

IV. Phân Chia Công Việc Cho 6 Người

Nguyên tắc: Mỗi người phụ trách 1–2 thuật toán + phần code/UI liên quan, đảm bảo khả năng và khó khăn khác nhau.

Thành viên 1 — Kiến trúc Backend & Game State

Hạng mục	Chi tiết
Thuật toán	Không có thuật toán AI riêng
File phụ trách	app.py (tổng cộng 150 dòng)
Nội dung công việc	Thiết kế RESTful API (/start, /move, /undo, /reset), quản lý Game State (board, move_history, game_over), tích hợp engine AI, xử lý validation đầu vào, cấu hình 3 mức độ khó (depth + time_limit)
Khả năng tính	★★★★ (Trung bình — nhiều logic quản lý trạng thái)

Thành viên 2 — Kiểm Tra Thắng & Sinh Nút Chơi

Hạng mục	Chi tiết
Thuật toán	(1) Check Winner, (4) Danger Detection, (5) Move Generation
File phụ trách	minmax.py — dòng 13–92
Nội dung công việc	Cài đặt check_winner(), is_dangerous(), get_possible_moves(). Giải thích cách quét 4 hàng, bản kính 2 ô, và phân loại must_block vs normal
Khả năng tính	★★★★ (Trung bình — 3 hàm, logic rõ ràng)

Thành viên 3 — Hàm Đánh Giá Thấp Cấp (Heuristic)

Hạng mục	Chi tiết
Thuật toán	(2) Pattern Counting, (3) Line Extraction, (6) Line Evaluation
File phụ trách	minmax.py — dòng 142–257
Nội dung công việc	Cài đặt _count_pattern(), _extract_all_lines(), _evaluate_line(). Thiết kế bảng pattern và trọng số. Giải thích chỉ số "Phòng thủ > Tấn công cấp 4"
Khả năng tính	★★★★ (Khó — bảng trọng số phức tạp, cần hiểu chi tiết thuật toán Caro)

Thành viên 4 — Đánh Giá Toàn Cục & Move Ordering

Hạng mục	Chi tiết
Thuật toán	(7) Global Evaluation, (8) Move Ordering
File phụ trách	minmax.py — dòng 96–139 và 260–292
Nội dung công việc	Cài đặt <code>evaluate()</code> (fork detection + center bonus), <code>score_move_quick()</code> (delta evaluation). Giải thích tại sao move ordering cần thiếp Alpha-Beta
Khải lược tính	★★★★ (Khó — cần hiểu sâu về cách diff ảnh hưởng tới sort order)

Thành viên 5 — Minimax, Alpha-Beta & Iterative Deepening

Hạng mục	Chi tiết
Thuật toán	(9) Iterative Deepening, (10) Minimax + Alpha-Beta Pruning
File phụ trách	minmax.py — dòng 295–380
Nội dung công việc	Cài đặt <code>minimax()</code> , <code>iterative_deepening()</code> , <code>TimeoutException</code> . Giải thích cây trò chơi, cắt tỉa alpha-beta, depth-adjusted win score, timeout mechanism
Khải lược tính	★★★★ (Rất khó — thuật toán trọng tâm của toàn bộ dự án)

Thành viên 6 — Frontend, UI/UX & Tích Hợp

Hạng mục	Chi tiết
Thuật toán	Không có thuật toán AI riêng
File phụ trách	game.js (280 dòng), index.html , style.css
Nội dung công việc	Thiết kế giao diện Glassmorphism/Dark Theme, render bàn cờ 15x15, xử lý sự kiện click, optimistic update (hiện quân O ngay trước khi server phản hồi), quản lý trạng thái UI (<code>isProcessing</code> , <code>gameActive</code>), hiển thị modal kết quả, hiển thị "AI đang suy nghĩ"
Khải lược tính	★★★★ (Trung bình — nhiều file nhưng logic UI không quá phức tạp)

Tổng hợp phân công:

Thành viên	Số thuật toán	Mức độ khó	File chính
1	0	★★★★	app.py
2	3	★★★★	minmax.py (L13–92)
3	3	★★★★	minmax.py (L142–257)
4	2	★★★★	minmax.py (L96–139, L260–292)
5	2	★★★★★	minmax.py (L295–380)
6	0	★★★★	game.js, HTML, CSS

V. Góc Nhìn Giảng Viên – Các Bài Tập Phân Tích Sâu & Câu Hỏi Về Đáp Án

Phần này mô phỏng góc nhìn của giảng viên phân tích trong buổi báo cáo của các sinh viên AI. Dưới đây là các bài tập phân tích sâu, kèm gợi ý trả lời.

Bài Tập 1 (Trọng tâm): Tại sao chọn Minimax thay vì Monte Carlo Tree Search (MCTS)?

Phân tích: Đây giúp nhận biết các chiến lược là câu hỏi ưu tiên của học sinh. Cần trả lời rõ ràng:

- Minimax + Alpha-Beta** phù hợp khi có thể xây dựng hàm đánh giá heuristic chất lượng cao — đúng với Gomoku vì hình thức pattern (3 hàng, 4 cột, fork...) đã được nghiên cứu kỹ và có thể lượng hoá chính xác.
- MCTS** phù hợp hơn cho các trò chơi mà hàm đánh giá khó xây dựng (như Go 19x19 có quá nhiều biến thể).
- Với bàn 15x15 và giới hạn thời gian 3s, Minimax + Alpha-Beta với depth 4–6 sẽ hiệu quả hơn. MCTS sẽ cần hàng nghìn simulation mới có thể đánh giá được.

Bài Tập 2 (Trọng tâm): Giảng viên thích cách nào để Alpha-Beta Pruning bằng ví dụ cụ thể

Phân tích: Giảng viên sẽ yêu cầu vẽ cây và giải thích khi nào cắt tỉa xảy ra. Chuẩn bị sẵn ví dụ:

```
MAX (3, 1) < MIN (0, MIN (0) < 3 < 3 (3) (3) (2) (2) <= Không cắt vì 2 < 3 (alpha=3)
```

- Node MAX đã biết nhánh trái cho giá trị ≥ 3 ($\alpha = 3$).
- Node MIN bên phải tìm được giá trị 2 $\rightarrow$ vì $2 < 3$ ($\beta \leq \alpha$), MAX sẽ không bao giờ chọn nhánh phải $\rightarrow$ cắt tỉa [?].

Bài Tập 3 (Trọng tâm): Bảng trạng thái pattern — cần sự lý thuyết hay thực nghiệm?

Phân tích: Đây là bài tập dành cho học viên. Câu trả lời trung thực:

- Bảng trình số xây dựng dựa trên **kinh nghiệm chiến thuật Gomoku** (các sách dạy cờ, bài nghiên cứu AI của Caro) kết hợp **hiệu chỉnh thực nghiệm** (chạy AI tự đấu với nhau, hiệu chỉnh cho đến khi AI phòng thủ tốt).
- Quyết định thi đấu quan trọng nhất: `_HU_HALF_4 = 1.500.000 > _AI_OPEN_4 = 1.000.000` — buộc AI phải chọn thắng khi tấn công. Nếu ngược lại, AI sẽ "tham" tấn công và thua trong các tình huống khi có 4 bước 1.

❖ 4: Move Ordering như thế nào bao nhiêu % hiệu năng?

Phân tích: Không có move ordering, Alpha-Beta hoạt động như Minimax thuần (worst case $O(b^d)$). Với move ordering tốt, đạt $O(b^{d/2})$ (best case). Trong thực tế:

- Move ordering tốt giúp cắt tỉa **70–90%** các nhánh.
- Thế hiện qua việc AI có thể đạt depth 6 trong 3 giây, trong khi không có move ordering chỉ đạt depth 3–4.

❖ 5: Tại sao cắt giới hạn 20 nước (moves[:20]) mà không phải 10 hay 30?

Phân tích: Đây là trade-off giữa **chất lượng** và **tốc độ**:

- 10 nước:** Quá ít, có thể bỏ sót nước đi quyết định (các biến nước chọn).
- 30 nước:** Quá nhiều, branching factor lớn, depth giảm.
- 20 nước:** Giá trị thực nghiệm (empirical), mức độ bao phủ các nước tấn công + phòng thủ quan trọng nhất mà vẫn cho phép search depth 4–6 trong 3 giây.

❖ 6: `depth * 1000` trong `WIN_SCORE` có ý nghĩa gì?

Phân tích: Kết thuật **Depth-adjusted Win Score** giúp quy định 2 ván:

- Mưu tiên thắng nhanh:** `WIN_SCORE + depth * 1000` — depth lớn hơn (gần gốc cây hơn) → giá trị lớn hơn → AI chọn con đường thắng ngắn nhất.
- Trì hoãn thua:** `-WIN_SCORE - depth * 1000` — khi thua là tất yếu, AI chọn con đường thua chậm nhất (depth nhỏ nhất), tạo cơ hội cho đối thủ mắc sai lầm.

❖ 7: Fork Detection — Tại sao phải xử lý riêng?

Phân tích: Fork (thắng gồng kìm) xảy ra khi AI/Human tạo được **≥ 2 mối đe dọa cấp cao đồng thời** (ví dụ: 2 chuỗi 3 hạt cùng lúc). Khi đó chỉ cần 1 nước → thua.

Pattern scoring nên thu được **không nên dựa vào mối quan hệ giữa các mối đe dọa đồng thời**. Fork bonus bù đắp thiếu sót này bằng cách thêm `100.000 + count * 30.000` khi có ≥ 2 threats.

❖ 8: Iterative Deepening có lãng phí không khi chạy lại depth 1, 2, 3... mãi lần?

Phân tích: Không đáng kể. Với branching factor b và max depth d : - Tổng nodes của tất cả depth 1..d: $b^1 + b^2 + \dots + b^d \sim b^d * b / (b-1)$ - Chi nodes của depth d : b^d - Overhead: $b / (b-1)$ khoảng 1.05–1.11 (chỉ thêm 5–11%)

Đặc biệt, ta cần **anytime behavior** (luôn có kết quả) và **time management** (không bao giờ vượt giờ hạn thời gian).

Milestone 15: TimeoutException — Tại sao dùng Exception thay vì flag kiểm tra?

Phân tích: Sử dụng Exception (raise `TimeoutException`) thay vì kiểm tra flag `if timeout: return` vì: - Minimax là hàm quy sâu nhiều tầng. Nếu dùng flag, mỗi tầng quy phải kiểm tra flag và truyền kết quả "bây giờ" ngược lên — code rất phức tạp. - Exception thoát trực tiếp qua tất cả các tầng quy về `iterative_deepening()`, nên nó bắt (except `TimeoutException`) một cách sạch sẽ. Kết quả `best_move` của depth hoàn chỉnh trước đó vẫn giữ nguyên.

Tài liệu được biên soạn ngày 10/06/2026 phục vụ buổi báo về dự án AI — Gomoku MinMax.